UNIVERSIDAD AUSTRAL

INGENIERÍA
Departamento de Computación

Sistemas Distribuidos

Clase 07 — Festival Austral: del checkout al log distribuido y recuperable

Departamento de Computación
Ingeniería Informática – Comisión A – Pilar
Equipo docente – 2026

Desafío de apertura: ¿qué es Kafka y por qué aparece tanto?

ACTIVIDAD

Tienen **tres minutos**: busquen en Google y preparen una respuesta breve para leer en voz alta.

1. ¿Qué es **Apache Kafka**?
2. ¿Para qué, cuánto y dónde en la realidad se usa kafka?

Caso conductor: CampusShop - la historia completa

NÚCLEO

Sofía — pedido 9F2A

`OrderCreated` → `StockReserved` → `PaymentAuthorized` → `OrderConfirmed`. Después, el servicio de entradas genera su código QR.

Tomás — pedido 1C7B

`OrderCreated` → `StockReserved` → `PaymentRejected` → `StockReleased` → `OrderCancelled`. Tomás no debe recibir QR.

QR — *Quick Response.* Código bidimensional que CampusShop usa como entrada digital al Festival Austral.

El accidente

El servicio que emite QR cae durante 40 segundos; Analítica sigue atrasándose mientras nuevas compras llegan.

Seguiremos estas dos historias de principio a fin. Cada concepto aparecerá mientras nos enfocamos en distintas fallas vinculadas a 9F2A o 1C7B.

Por qué dividimos la historia en cinco capítulos

NÚCLEO

Capítulo	Problema de la historia	Conceptos que necesitaremos
1. Publicar	El checkout no debe esperar a QR y Analítica, pero el hecho debe sobrevivir.	productor, broker, log, Kafka, ACK
2. Ordenar	Los estados de cada pedido deben conservar una secuencia sin serializar todas las compras.	topic, partición, key, offset, hot partition
3. Consumir	QR cae después de actuar y otra instancia debe continuar sin perder ni duplicar.	poll, grupo, committed offset, replay, inbox, rebalanceo, lag
4. Publicar seguro	La base puede confirmar el pedido y el proceso caer antes de publicar.	WAL local, outbox, relay, CDC, Debezium, frontera exactly-once
5. Operar el stream	Analítica recibe una historia ilimitada, con atraso y eventos fuera de orden.	stream, pipeline, estado, event time, ventana, watermark

La división no es un índice de productos: es la cronología que pueden llevar a fallas en un sistema distribuido.

Qué significa “compra confirmada” en esta clase

NÚCLEO

Antes de mostrar el OK

- pedido durable;
- stock reservado;
- pago autorizado;
- intención durable de publicar `OrderConfirmed`.

Después del OK

- emitir el código QR;
- enviar una notificación;
- actualizar tablero, ranking e historial.

Esas lecturas posteriores son **vistas derivadas** o proyecciones: representan un hecho ya confirmado; no deciden si la compra fue válida.

Semántica elegida

Si el pago de Tomás falla, el pedido se cancela, la reserva se libera y no existe `OrderConfirmed(1C7B)`.

Checkout: qué es y qué no es

Checkout es el servicio o flujo de aplicación que recibe la intención de compra, coordina los pasos críticos y registra el resultado del pedido.

- Recibe la petición del usuario.
- Usa servicios de Pago e Inventario.
- Persiste el estado del pedido.
- Responde cuando se cumple la promesa elegida.

No es lo mismo que...

- la base de datos;
- el WAL del motor;
- el broker de eventos;
- el servicio de QR.

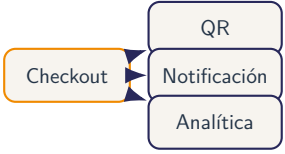
Checkout solicita una transacción; el motor de la base usa su WAL para recuperar el commit local.

De llamadas directas a una historia compartida

NÚCLEO

Fan-out sincrónico

Una petición se ramifica en varias llamadas y Checkout espera las respuestas antes de terminar.



Problema

El componente más lento fija la latencia; una dependencia secundaria puede hacer fallar una compra válida.

Historia compartida y preservada

Checkout publica una vez `OrderConfirmed(9F2A)`. Cada lector avanza y se recupera de forma independiente.



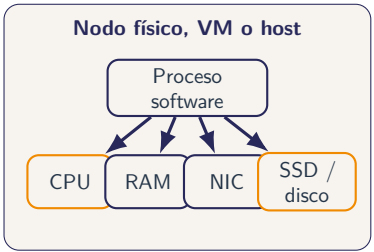
El trabajo sale del camino crítico y puede recuperarse sin que Checkout siga vivo.

Base mínima I: quiénes participan

NÚCLEO

Actor	Definición de trabajo
Usuario / cliente	Persona o aplicación que inicia una petición.
Balanceador de carga	Distribuye peticiones entre varias instancias del servicio.
Instancia de API	Proceso que recibe una petición y ejecuta lógica de aplicación.
Productor	Código o proceso que publica un registro en el log distribuido.
Broker	Proceso servidor que aloja particiones, conserva registros y atiende productores y consumidores.
Consumidor	Proceso que lee registros y produce un efecto o estado derivado.
Servicio de QR	Consumidor que genera y conserva una entrada digital identificable.

Base mínima II: cómo un proceso usa los recursos de un nodo



CPU: uno o más núcleos ejecutan el código del proceso: validación, serialización, compresión o lógica.

RAM: conserva buffers, lotes, cachés y estado temporal.

NIC: interfaz que envía y recibe tráfico de red.

Medio persistente: SSD, NVMe o disco que conserva segmentos y estado durable.

Base mínima III: comando, evento, registro y log

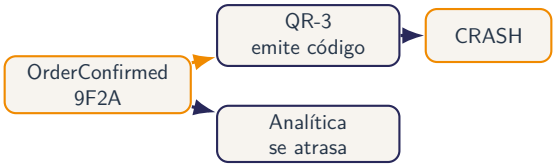
NÚCLEO

Término	Definición y ejemplo
Comando	Intención que puede aceptarse o rechazarse: `ConfirmOrder(9F2A)`.
Evento	Hecho que el dominio considera ocurrido: `OrderConfirmed(9F2A)`.
Registro	Representación almacenada del hecho: identidad, key, tipo, datos y metadatos.
Append	Agregar un registro al final de una partición.
Log	Secuencia ordenada y preservada de registros que puede recorrerse y releerse.
Replay	Volver a leer registros para reconstruir procesamiento o estado.

El evento es el significado; el registro es su representación almacenada; el log es la secuencia de esos registros.

El accidente de las 12:00:02

PREDICCIÓN



Predicción

¿Cómo recupera QR el trabajo? ¿Qué historia, marca de progreso e identidad deben conservarse para no depender de que Checkout siga vivo?

Guarde su respuesta: la cerraremos en el Capítulo 3 con `poll`, committed offset, replay e inbox.

Pregunta central: del hecho al efecto recuperable

NÚCLEO

¿Cómo convertir dos compras concurrentes en historias compartidas que distintos componentes puedan procesar y reprocesar sin perder progreso ni repetir el efecto lógico?



Evidencias de aprendizaje esperadas

NÚCLEO

Al finalizar, debemos poder entender:

1. elegir una `key` y explicar qué orden conserva y qué carga concentra;
2. mover un crash en el timeline y distinguir pérdida, repetición y replay seguro;
3. explicar cómo un consumidor reinicia desde progreso durable;
4. diseñar outbox e inbox y declarar la frontera de una garantía exactly-once.

La evidencia será un diagrama, una decisión y una explicación breve; no una lista memorizada de componentes de Kafka.

Diagnóstico: razonemos estas preguntas

ACTIVIDAD

1. ¿Los estados de una misma compra deben conservar un orden?
2. ¿Las compras de Sofía y Tomás necesitan un orden global único?
3. Si QR cae después de emitir el código, ¿qué debe reaparecer al reiniciar?
4. Si la base guardó la compra pero el hecho no se publicó, ¿cómo se enteran QR y Analítica?
5. Si una partición acumula trabajo, ¿agregar consumidores siempre la divide?

Puente desde la clase 6: tres historias distintas

NÚCLEO

Objeto	Quién lo administra	Para qué sirve
WAL local	motor de la base	recuperar páginas y transacciones del nodo
Outbox	aplicación + base local	conservar, junto con el negocio, la intención de publicar
Log distribuido	cluster de brokers	preservar y compartir registros para múltiples consumidores

Checkout no "ejecuta el WAL": solicita una transacción; el motor usa WAL para el commit local. La outbox enlaza ese commit con el log distribuido, que otros procesos pueden leer y releer.

Actividad: ¿qué ocurre antes del OK y qué puede ocurrir después?

ACTIVIDAD

Clasifique cada efecto de 9F2A:

Efecto	¿Antes del OK?	Razón
Pedido durable		
Reserva de stock		
Autorización de pago		
Intención durable de publicar		
Emitir QR		
Actualizar analítica / ranking		

Respuesta: qué debe ocurrir antes y después del OK

NÚCLEO

Efecto	Momento	Justificación
Pedido, reserva y pago	antes del OK	hacen verdadera la promesa “compra confirmada”
Intención durable de publicar	antes del OK	evita confirmar sin una salida recuperable
QR, notificación y analítica	después del OK	pueden demorarse o reintentarse sin invalidar la compra

Para `1C7B`, el pago es rechazado: antes de responder deben quedar registradas la liberación de la reserva y la cancelación. No debe emitirse QR.

Camino crítico: lo que el usuario debe esperar

Definición

El **camino crítico de la petición** es la secuencia de pasos que debe terminar antes de poder responder correctamente al usuario. Su duración condiciona la latencia del Checkout.



Criterio

Si la caída de un paso vuelve falsa la frase “compra confirmada”, ese paso pertenece al camino crítico.

Efectos asincrónicos: necesarios, pero fuera de la espera

NÚCLEO

Qué significa asincrónico aquí

Checkout no espera que el efecto termine para responder. El efecto sigue siendo necesario, pero puede ejecutarse después, reintentarse y recuperarse de forma independiente.



Asíncrono no significa prescindible. Significa que el efecto no determina la validez de la compra y puede progresar a su propio ritmo.

UNIVERSIDAD AUSTRAL

INGENIERÍA

Departamento de Computación

Capítulo 1 — publicar el hecho sin esperar a todos

Sistemas Distribuidos

Capítulo 1: dónde estamos en la historia

NÚCLEO

Sofía ya recibió “compra confirmada”. QR y Analítica todavía no terminaron y no deben mantener bloqueado al checkout.

Necesidad

Conservar `OrderConfirmed(9F2A)` fuera de la memoria del productor y permitir que varios lectores lo procesen a ritmos distintos.

Pregunta del capítulo

¿Qué actores publican, conservan y leen el hecho, y qué significa que el broker lo haya confirmado?

Arquitectura mínima: productor, brokers y consumidores

NÚCLEO



El productor agrega un registro a una historia compartida y persistida; cada aplicación consumidora conserva su propio progreso y puede recuperarse de manera independiente.

Kafka: una implementación real del modelo

CASO REAL

Apache Kafka es una plataforma distribuida de *event streaming*: productores publican registros, brokers los conservan en logs particionados y consumidores los leen con progreso propio.

Qué problema resuelve aquí

Desacoplar el checkout de QR y Analítica, conservar el hecho y permitir replay después de una caída.

Qué no resuelve automáticamente

No decide la key, no hace idempotente el QR, no elimina dual write y no define la semántica de “compra confirmada”.

Del software al hardware: recorrido mínimo de un evento

NÚCLEO

Nodo	Proceso	Recursos utilizados
Aplicación	productor	el código ejecuta en CPU; RAM forma el batch; NIC lo envía
Broker líder	broker	NIC recibe; CPU valida; RAM/page cache bufferiza; SSD o disco conserva segmentos
Broker seguidor	réplica	NIC recibe la copia; CPU verifica; medio persistente la conserva
Consumidor	lector	NIC recibe lote; RAM bufferiza; CPU ejecuta lógica; DB/SSD conserva el efecto

"Publicar un evento" se materializa como instrucciones, buffers, paquetes y escrituras en varios nodos. El proceso usa los recursos; la CPU no es el productor ni el broker.

UNIVERSIDAD AUSTRAL

INGENIERÍA

Departamento de Computación

Capítulo 2 — ordenar cada compra sin ordenar el mundo entero

Sistemas Distribuidos

Capítulo 2: dónde estamos en la historia

NÚCLEO

El hecho ya puede publicarse. Ahora debemos mantener coherente el ciclo de vida de cada pedido sin obligar a que todas las compras del Festival formen una sola fila global.

Pregunta

¿Qué eventos deben permanecer juntos porque uno depende del anterior?

Decisión

Elegir una key, una partición y un ámbito de orden compatible con causalidad y carga.

Vocabulario previo del capítulo: topic y partición

NÚCLEO

Topic: categoría lógica de registros con un contrato común; usaremos `orders.lifecycle`.

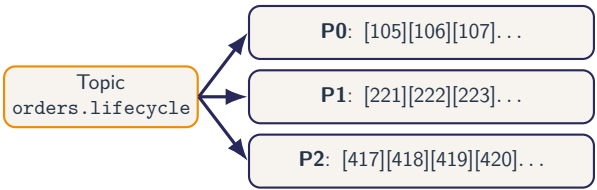
Partición: uno de los logs ordenados que componen el topic. Se almacena, replica y consume como unidad.

Límite

El orden existe dentro de cada partición. No aparece un orden global gratuito entre particiones diferentes.

P0, P1 y P2: tres particiones del mismo topic

NÚCLEO



Los offsets se interpretan dentro de una partición. `P0:107` y `P2:419` no forman un orden global entre sí.

Consideremos las dos historias

NÚCLEO

Compra 9F2A — Sofía

1. `OrderCreated`
2. `StockReserved`
3. `PaymentAuthorized`
4. `OrderConfirmed`

El QR será una consecuencia posterior de la confirmación y podrá recuperarse desde el log.

Compra 1C7B — Tomás

1. `OrderCreated`
2. `StockReserved`
3. `PaymentRejected`
4. `StockReleased`
5. `OrderCancelled`

Pregunta

¿Qué relaciones causales deben conservarse dentro de cada compra y qué eventos de compras distintas pueden intercalarse?

Causalidad necesaria: qué orden no podemos perder

NÚCLEO



La **key no crea causalidad**: reúne en una partición los eventos cuyo orden debe preservarse. El productor o dueño del agregado debe emitirlos coherentemente; un campo `aggregate_seq` puede detectar huecos, repetición o desorden.

- `OrderConfirmed(9F2A)` presupone reserva y pago autorizados.
- `OrderCancelled(1C7B)` presupone rechazo y liberación de stock.
- 9F2A y 1C7B pueden intercalarse: son historias independientes.

Qué es una key de particionamiento y qué no hace

NÚCLEO

La **key** es un valor estable extraído del registro. El productor aplica una función determinista para seleccionar una partición.

$$\text{partición} = h(\text{key}) \bmod P$$

Sí hace

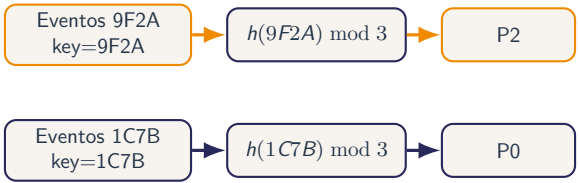
Mantiene la misma key en una misma partición y habilita orden local entre sus appends.

No hace

No valida el orden del negocio, no evita hot keys y no crea orden global entre particiones.

Elección para la historia: key = order_id

NÚCLEO



No elegimos manualmente la partición para cada evento: elegimos una key estable y la función produce siempre el mismo destino. Los valores P0 y P2 son ilustrativos para esta historia.

Las dos historias ya ubicadas en tres particiones

Partición	Registros en orden de append y offset
P0	105:Created(1C7B) → 106:StockReserved → 107:PaymentRejected → 108:StockReleased → 109:Cancelled
P1	*Ningún registro de las dos compras analizadas cae aquí. Otras keys podrían usarla en producción.*
P2	417:Created(9F2A) → 418:StockReserved → 419:PaymentAuthorized → 420:Confirmed

Resultado preciso

Todos los registros de `1C7B` permanecen juntos en P0 y todos los de `9F2A` en P2. La partición conserva su orden de append; `aggregate_seq` permite comprobar el ciclo de vida esperado.

Anatomía de un registro de la historia 9F2A

NÚCLEO

Campo	Ejemplo y función
`event_id`	`evt-9F2A-04`: identidad única para deduplicación
`aggregate_id`	`9F2A`: key que agrupa la historia del pedido
`aggregate_seq`	`4`: posición causal dentro del ciclo de vida
`type`	`OrderConfirmed`: significado del hecho
`partition/offset`	`P2:420`: posición del registro dentro del log

Identidad de evento, identidad del pedido, secuencia del agregado y offset responden preguntas distintas; no son intercambiables.

Qué ganamos y a qué renunciamos

NÚCLEO

Ganamos

- orden por pedido;
- paralelismo entre pedidos;
- distribución de escritura y consumo;
- recuperación por partición.

Renunciamos

- orden global entre todas las compras;
- procesamiento paralelo dentro de una misma key;
- rebalancear una key sin mover su historia.

La partición es simultáneamente una frontera de orden, capacidad y paralelismo.

Qué hace buena a una key

NÚCLEO

Criterio	Pregunta de diseño
Causalidad	¿Agrupa exactamente los eventos cuyo orden relativo importa?
Estabilidad	¿El valor permanece igual durante toda la vida de la historia?
Cardinalidad	¿Existen suficientes valores distintos para repartir la carga?
Granularidad	¿Evita serializar eventos que no necesitan esperar entre sí?
Distribución	¿Ningún valor concentra una fracción desproporcionada del tráfico?
Observabilidad	¿Podemos medir bytes, QPS y lag por key/partición?

Una buena key es el identificador estable más pequeño que preserva la causalidad necesaria y distribuye historias independientes.

Tres keys posibles para el mismo evento

Key	Efecto sobre el orden	Efecto sobre la carga
`order_id`	conserva el ciclo de vida del pedido	muchos pedidos se reparten
`event_id`	eventos del mismo pedido pueden separarse	excelente reparto, causalidad rota
`campus_id`	ordena todo el campus	una venta masiva concentra todo

No existe una key universal

La key depende del invariante y del orden que necesita ese topic. Otra vista puede requerir otro topic o una proyección derivada.

Hot key y hot partition: una key concentra la carga

EXTENSIÓN

Una **hot key** produce mucho más tráfico que las demás. Como una key pertenece a una sola partición, puede convertirla en una **hot partition**.

- `order_id`: muchos pedidos independientes suelen repartirse bien.
- `campus_id`: toda la venta del Festival termina en una única partición.
- Una entidad individual muy activa no puede paralelizarse sin redefinir qué orden necesita.

Opciones

Rediseñar la key, dividir la entidad si la semántica lo permite, aceptar el cuello de botella o agregar resultados parciales en otra etapa.

Agregación en dos etapas — también llamada jerárquica

EXTENSIÓN

Idea

En vez de enviar todos los eventos a una única key global, cada partición calcula un resultado parcial y una segunda etapa combina los parciales.

$$\text{ventas P0} + \text{ventas P1} + \text{ventas P2} = \text{ventas totales}$$

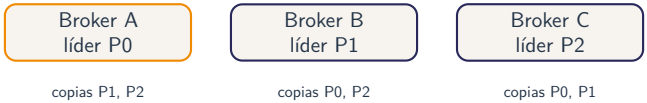
Sirve para conteos, sumas, mínimos, máximos e histogramas combinables.

Límite

No sirve para dividir arbitrariamente la historia ordenada de `9F2A`.

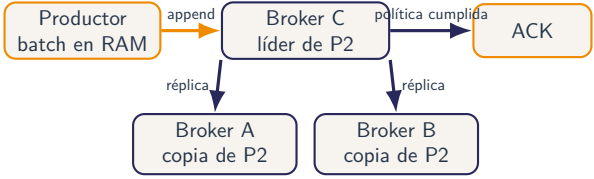
De particiones lógicas a brokers físicos

NÚCLEO



- P0 contiene la historia 1C7B; su líder atiende appends y lecturas.
- P2 contiene la historia 9F2A; su líder está en otro nodo.
- Las copias protegen frente a pérdida de nodo, pero consumen red y disco.

Publicar Confirmed(9F2A): del productor al ACK



El líder responde cuando se cumple la política configurada de confirmación. El ACK no significa automáticamente "un `fsync` independiente por evento".

Timeout del productor: el evento 420 puede o no existir

NÚCLEO

Historia A

`Confirmed(9F2A)` nunca llegó a P2.
Reintentar es necesario.

Historia B

P2 agregó el evento en offset 420, pero se perdió el ACK. Reintentar puede duplicarlo.

La idempotencia del productor usa identidad y secuencia para que el broker reconozca reenvíos del mismo batch. No deduplica el cobro ni el email externo.

UNIVERSIDAD AUSTRAL

INGENIERÍA

Departamento de Computación

Capítulo 3 — QR cae: otro lector debe continuar

Sistemas Distribuidos

Capítulo 3: dónde estamos en la historia

NÚCLEO

`OrderConfirmed(9F2A)` quedó en P2:420. El servicio QR emitió la entrada digital y cayó antes de registrar cuánto había avanzado.

Problema

Otra instancia debe continuar sin perder la confirmación ni emitir dos QR lógicos.

Conceptos que nacen aquí

`poll`, position, committed offset, replay, consumer group, inbox, rebalanceo y lag.

Vocabulario previo del capítulo: leer y guardar progreso

NÚCLEO

Término	Significado en la historia
`poll`	llamada con la que QR solicita al broker un lote de registros
Polling	repetición de esa consulta mientras la aplicación continúa viva
Position	próximo offset que la instancia actual intentará leer
Committed offset	progreso durable desde el que otra instancia puede reiniciar
Replay	relectura de registros ya vistos por una instancia anterior

Un ciclo de consumo de QR

NÚCLEO

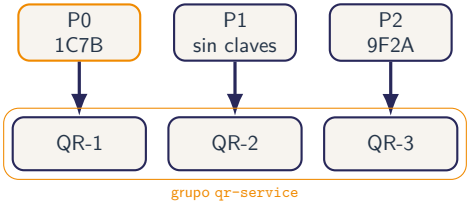


El broker entrega registros; QR decide el efecto. Guardar offset 421 significa que la próxima instancia comenzará después de P2:420.

Consumer group: una aplicación lógica con varias instancias

Definición

Un **consumer group** es un conjunto de procesos que ejecutan la misma aplicación consumidora y comparten un identificador, `group.id`. El broker reparte **particiones completas** entre los miembros activos.



Dentro de un grupo, cada partición tiene una sola instancia activa. Una instancia puede recibir varias; las instancias sobrantes quedan ociosas.

Dos aplicaciones pueden leer la misma historia

NÚCLEO

Grupo QR

`group.id = qr-service`

- P0 → QR-1
- P1 → QR-2
- P2 → QR-3

Genera entradas digitales y conserva su progreso por partición.

Grupo Analítica

`group.id = festival-analytics`

- también lee P0, P1 y P2;
- mantiene offsets independientes;
- puede estar atrasado aunque QR esté al día.

Las particiones se reparten entre instancias del **mismo grupo**; no se reparten entre todas las aplicaciones del sistema.

Las dos historias vistas por el grupo QR

NÚCLEO

QR-1 lee P0

1C7B termina en `OrderCancelled`. El consumidor registra que no debe emitir entrada.

QR-3 lee P2

9F2A alcanza `OrderConfirmed`. El consumidor genera una entrada QR identificada por el pedido.

Consecuencia

Los pedidos se procesan en paralelo; el efecto depende del tipo de evento, no solo de haber leído un registro.

El accidente de QR-3

PREDICCIÓN

QR-3 lee `OrderConfirmed(9F2A)` en P2:420, persiste el código de entrada y cae antes de guardar su committed offset.



Predicción

¿Qué reaparece y cómo evitamos crear dos entradas válidas para la misma compra?

At-most-once: progreso antes que efecto

poll 420 $\rightarrow$ commit 421 $\rightarrow$ crash $\rightarrow$ efecto nunca aplicado

Resultado

El evento no reaparece, pero se puede perder la emisión del QR.

At-most-once prioriza no repetir; una falla entre progreso y efecto puede producir pérdida lógica.

At-least-once: efecto antes que progreso

poll 420 → emitir QR → crash → replay 420

Resultado

El evento reaparece; si el efecto no es idempotente, pueden emitirse dos QR válidos.

At-least-once facilita progreso frente a fallas, pero exige idempotencia o deduplicación del efecto.

Inbox idempotente: recordar el evento junto con el efecto

NÚCLEO



Si 420 reaparece, la inbox demuestra que el QR ya fue emitido. Identidad y efecto deben confirmarse atómicamente.

Actividad: mueva el crash en la historia 9F2A

ACTIVIDAD

Ubique el crash en cuatro cortes:

1. antes de emitir QR;
2. después de emitir QR, antes de inbox;
3. después de inbox, antes de committed offset;
4. después del committed offset.

Para cada corte, responda:

- ¿qué estado es durable?;
- ¿qué offset reaparece?;
- ¿se pierde o duplica el efecto?;
- ¿qué debe ser atómico?

Por qué aparecen rebalanceo y lag en esta historia

NÚCLEO

1. QR-3 era responsable de P2 y cae después de emitir el QR de `9F2A`.
2. El grupo debe asignar P2 a otra instancia para continuar.
3. Mientras cambia la asignación, P2 sigue recibiendo nuevos registros.

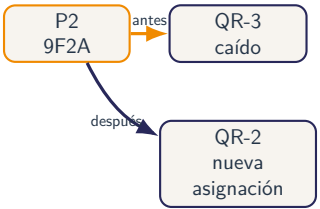
Rebalanceo: ¿quién procesa ahora P2?

Lag: ¿cuánto trabajo quedó pendiente mientras P2 no avanzaba al ritmo de ingreso?

Son dos consecuencias del mismo accidente: reasignar responsabilidad y medir la deuda acumulada.

Rebalanceo: QR-3 cae y P2 cambia de dueño

NÚCLEO



Rebalanceo: el grupo detecta cambios de membresía y recalcula qué instancia procesa cada partición.

- el consumo puede pausarse durante el cambio;
- QR-2 retoma P2 desde el committed offset;
- la asignación puede volver a cambiar ante otra falla.

Lag: la deuda de procesamiento de P2

Por qué aparece

Durante el rebalanceo y la recuperación, el broker sigue aceptando nuevos registros en P2. Si ingresan más registros por segundo de los que QR-2 procesa, crece el trabajo pendiente.

$$L_o = O_{\text{final de P2}} - O_{\text{confirmado por QR}}$$

Lag en offsets: cantidad aproximada de registros pendientes.
Lag temporal: edad de la compra confirmada más antigua que todavía no obtuvo QR.

Interpretación

El lag no es la falla original: es la señal observable de que producción y consumo avanzan a ritmos diferentes.

Dashboard por partición: no diagnostique solo el promedio

EXTENSIÓN

Partición	Ingreso/s	Proceso/s	Lag	CPU	Hipótesis
P0	220	240	0	45%	sana
P1	210	230	12	48%	fluctuación
P2	520	260	19.400	91%	carga/skew/restore

Una hot partition, un restore o una dependencia base de QR lenta pueden producir el mismo síntoma superficial: lag creciente.

UNIVERSIDAD AUSTRAL
INGENIERÍA
Departamento de Computación

Capítulo 4 — el pedido existe: el evento también debe existir

Sistemas Distribuidos

Capítulo 4: dónde estamos en la historia

NÚCLEO

Ya sabemos publicar, particionar y recuperar consumidores. Falta una falla anterior: la base de Orders puede confirmar 9F2A y el proceso caer antes de que exista `OrderConfirmed` en el broker.

Riesgo

El usuario ve OK, pero QR y Analítica jamás reciben el hecho.

Necesidad

Unir el commit local y la intención de publicar sin depender de dos escrituras remotas atómicas.

La nueva falla: pedido confirmado, evento ausente

Checkout confirma 9F2A en la base y luego intenta publicar `OrderConfirmed`.



Consecuencia

La compra existe, pero QR y Analítica nunca reciben el hecho.

Dual write: dos escrituras independientes para un mismo hecho

NÚCLEO

Definición

Existe **dual write** cuando una operación de negocio necesita modificar dos sistemas que no comparten una transacción atómica; aquí, la base de pedidos y el broker de eventos.

Base primero

DB confirma → crash → la compra existe, pero el evento falta.

Broker primero

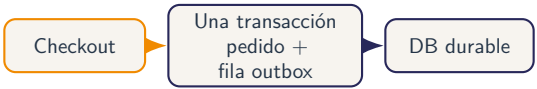
Evento publicado → crash o rollback → los consumidores ven una compra que no quedó confirmada.

Necesitamos el mismo resultado lógico —pedido confirmado y hecho publicable—, pero no existe un COMMIT único que abarque ambas escrituras remotas.

Transactional outbox: el pedido y su evento nacen juntos

NÚCLEO

Una **outbox transaccional** es una tabla de la misma base donde Checkout guarda el evento pendiente. La fila outbox se inserta en el **mismo COMMIT local** que confirma el pedido.



Si el COMMIT existe, existen juntos el negocio y la intención de publicar. Si aborta, no existe ninguno.

Qué deja abierto

Otro proceso debe publicar la fila. Puede repetirla, por lo que los consumidores posteriores necesitan identidad e idempotencia.

Relay, polling y CDC: cómo sale el evento de la outbox

NÚCLEO

Término	Definición
Relay	Proceso que toma filas pendientes de la outbox y las publica en el broker.
Polling	El relay consulta periódicamente la tabla outbox para encontrar trabajo.
CDC	*Change Data Capture*: captura cambios ya confirmados en una base.
WAL / binlog	Registros internos desde los cuales una herramienta CDC puede observar esos cambios.
Downstream	Componentes ubicados más adelante en el flujo, como QR o Analítica, que reciben y procesan el evento.

Límite

El relay puede publicar y perder el ACK; al reintentar, el broker puede recibir otra copia. La outbox evita "DB sí / evento no", pero no elimina duplicados en los consumidores posteriores.

Debezium: un relay basado en CDC

CASO REAL

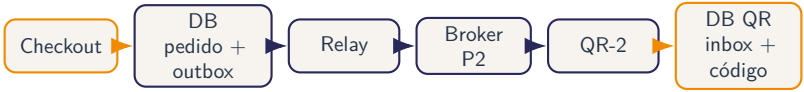
Debezium es una plataforma de conectores CDC. Puede leer cambios confirmados desde WAL o binlog y transformarlos en eventos publicados, frecuentemente en Kafka.



Debezium es una implementación posible del relay; la idea arquitectónica es la outbox durable más publicación repetible.

Flujo completo de 9F2A: outbox, broker e inbox

NÚCLEO



- Outbox vuelve durable la intención de publicar.
- El broker conserva y permite replay.
- Inbox vuelve idempotente la emisión del QR.
- El mismo `event_id` conecta el recorrido extremo a extremo.

Exactly-once: declare el objeto y la frontera

NÚCLEO

"Exactly-once" no es una propiedad universal. Debe especificar qué estado, outputs y progreso se confirman juntos.

Dentro de la frontera

Input, estado local, output y progreso pueden participar en una transacción coordinada.

Fuera de la frontera

Email, pago o API externa requieren identidad, idempotencia, deduplicación o reconciliación propia.

UNIVERSIDAD AUSTRAL
INGENIERÍA
Departamento de Computación

Capítulo 5 — Analítica debe operar una historia que nunca termina

Capítulo 5: dónde estamos en la historia

NÚCLEO

Pedidos y QR ya son recuperables. Analítica lee el mismo log con otro grupo y debe producir el tablero de ventas del Festival mientras siguen llegando eventos atrasados.

Pregunta

¿Cómo convertir una secuencia ilimitada en resultados finitos por minuto o por ventana?

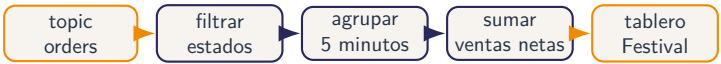
Nueva dificultad

La hora de ocurrencia y la hora de procesamiento pueden ser diferentes.

Stream y pipeline: Analítica procesa la venta del Festival

NÚCLEO

Stream: secuencia potencialmente ilimitada de eventos.
Pipeline: etapas que filtran, transforman, agrupan o enrutan esos eventos.



Stateless y stateful: qué debe recordar Analítica

NÚCLEO

Stateless

Transforma cada registro aislado: normalizar moneda, validar esquema o seleccionar campos.

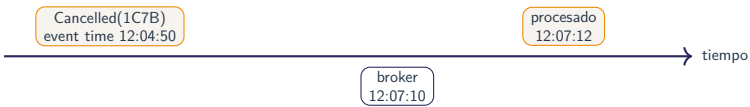
Stateful

Mantiene información entre eventos: ventas confirmadas menos cancelaciones por ventana.

El estado puede vivir en RAM para velocidad, en SSD local para recuperación rápida y en un changelog remoto para reconstrucción si se pierde el nodo.

Event time y processing time: 1C7B llega tarde

Event time: cuándo ocurrió el hecho en el dominio.
Processing time: cuándo lo procesa Analítica.



Consecuencia

Si la ventana 12:00–12:05 ya fue publicada, Analítica debe corregirla, reabrirla, descartar el evento o enviarlo a una salida lateral según la política elegida.

Watermark: estimar cuánto avanzó el tiempo de los eventos

NÚCLEO

Problema

Analítica no puede esperar para siempre por un evento antiguo, pero llegada y ocurrencia no tienen el mismo orden.

Una estrategia común calcula, por partición, el mayor *event time* observado menos un margen de desorden permitido Δ:

$$W_i = \max(\textit{event time en } P_i) - \Delta.$$

El progreso global queda limitado por la partición activa más atrasada:

$$W = \min_i W_i.$$

Lectura

Si $W > 12{:}05$, se considera razonable cerrar 12:00–12:05. Es una política de espera, no una promesa de que jamás llegará algo anterior.

Ejemplo de watermark para la ventana 12:00–12:05

NÚCLEO

Suponga un margen de desorden permitido de $\Delta = 2$ minutos:

Partición	Mayor event time observado	Watermark local
P0	12:07:20	12:05:20
P1	12:07:10	12:05:10
P2	12:06:50	12:04:50

$$W = \min(12{:}05{:}20, 12{:}05{:}10, 12{:}04{:}50) = 12{:}04{:}50$$

Decisión

Todavía no se cierra la ventana: P2 mantiene el progreso global antes de 12:05. Cuando P2 avance lo suficiente, W superará 12:05. Si después llega `Cancelled(1C7B)` con *event time* 12:04:50, será *late data*.

Una partición sin eventos puede marcarse temporalmente como inactiva; de lo contrario, podría frenar el watermark global de forma indefinida.

SLO: cuánto atraso estamos dispuestos a aceptar

NÚCLEO

SLO — *Service Level Objective*, u objetivo de nivel de servicio: meta cuantificable que el sistema intenta cumplir durante un período.

- QR: "el 99% de las compras confirmadas recibe su código en menos de 30 segundos".
- Analítica: "el lag temporal p95 permanece por debajo de 60 segundos".
- Ranking: "las posiciones visibles tienen menos de 10 segundos de atraso durante el 99% de la hora pico".

El SLO convierte "debe ser rápido" en una condición medible para dimensionar particiones, consumidores, rebalances y políticas de late data.

Transferencia: ranking de Asteria Online

ACTIVIDAD

Ahora transfiera el método fuera del Festival:

1. evento y `event_id`;
2. key y orden necesario;
3. número inicial de particiones;
4. estado que conserva el consumidor;
5. política de late data;
6. SLO de atraso —objetivo medible y ventana de evaluación—;
7. prueba de crash, replay y rebalanceo.

Pregunta de transferencia

¿Qué cambia si el ranking entrega premios irreversibles al cerrar la temporada?

La historia completa de 9F2A y 1C7B

NÚCLEO

1. Checkout coordina los pasos críticos y guarda pedido + intención de publicar.
2. Relay publica eventos con `key=order_id`.
3. 9F2A llega a P2 y 1C7B a P0 con secuencia de agregado verificable.
4. QR y Analítica leen la misma historia en grupos independientes.
5. Si QR-3 cae, QR-2 retoma P2 desde el committed offset.
6. Inbox evita dos QR lógicos ante replay.
7. Analítica calcula ventanas por event time y corrige late data según política.

Cada concepto respondió a una falla concreta de las mismas dos compras.

Método reusable para diseñar un pipeline

NÚCLEO

Para cada flujo, responda en este orden:

1. ¿Cuál es el hecho y cuál su identidad estable?
2. ¿Qué eventos requieren orden relativo?
3. ¿Qué key agrupa esa causalidad sin concentrar carga?
4. ¿Qué particiones y consumidores necesita el throughput?
5. ¿Dónde se guarda el progreso recuperable?
6. ¿Qué efecto debe ser idempotente?
7. ¿Existe dual write? ¿Outbox/inbox?
8. ¿Qué métricas detectan skew, lag y rebalances?

Volvemos al Diagnóstico inicial

ACTIVIDAD

Después de entender lo anterior podríamos responder con criterio:

1. ¿Por qué `order_id` reúne la historia y qué condición adicional preserva la causalidad?
2. ¿Por qué `campus_id` puede producir una hot partition?
3. ¿Qué reaparece si el QR fue durable pero el committed offset no?
4. ¿Qué resuelve outbox y qué deja abierto?
5. ¿Por qué agregar consumidores no divide una partición?

Síntesis: cinco ideas que deben quedar

NÚCLEO

1. La key reúne el ámbito de orden; el productor y la secuencia del agregado deben representar correctamente la causalidad.
2. Una partición ofrece orden local y es unidad de almacenamiento y paralelismo.
3. Committed offset registra progreso; no confirma por sí solo el efecto de negocio.
4. Replay es normal; idempotencia e inbox vuelven seguro el efecto.
5. Outbox conecta una transacción local con publicación repetible, no con exactly-once universal.

Exit ticket

ACTIVIDAD

En una hoja, responda:

1. Defina una buena key en una frase y aplíquela a 9F2A.
2. Dibuje dónde quedan 9F2A y 1C7B en tres particiones.
3. Explique qué ocurre si QR cae después de emitir la entrada y antes del committed offset.
4. Nombre una métrica que distinga hot partition de atraso general.

Fuentes y material de respaldo

RESPALDO

- Apache Kafka Documentation: topics, partitions, producer, consumer groups y delivery semantics.
- Kreps et al.: *Kafka: a Distributed Messaging System for Log Processing*.
- Kleppmann: *Designing Data-Intensive Applications*, capítulos sobre logs, particionado y stream processing.
- Debezium Documentation: transactional outbox y CDC.
- Apache Flink Documentation: event time, watermarks y stateful processing.

Lea siempre la documentación buscando cinco preguntas: orden, progreso, falla, efecto y costo operacional.